

Agent Runtime 技术全景

Agent Runtime – Planner 子领域技术调研

产出日期: 2026-07-07 | 覆盖项目: **gpt-researcher** | 研究员: Claude (planner subdomain)

1. 子领域定位：Planner 在 Agent Runtime 中的位置

Agent Runtime 的八个子领域（sandbox / memory / gateway / planner / observability / protocol / security / tool）中，**planner 是”大脑中枢”**——它决定 Agent “先做什么、后做什么、用什么做”。如果说 memory 是存储证据的档案室、sandbox 是执行动作的隔离车间、gateway 是统一入口，那么 **planner 就是下达调度指令的总指挥**。

Planner 需要解决的核心问题可归纳为四个：

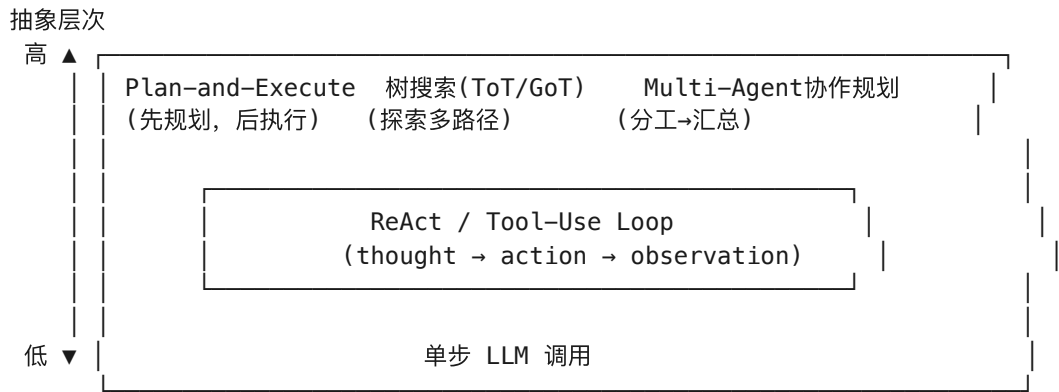
问题	描述
任务分解 (Task Decomposition)	给定复杂的用户目标，如何拆分出可执行的子任务序列？
工具选择与调度 (Tool Selection & Orchestration)	给定一组可用工具/能力，何时调用哪个、传入什么参数？
上下文管理与记忆读写 (Context & Memory)	多步执行中，哪些中间结果需要保留？何时读、何时写 memory？
错误恢复与重规划 (Error Recovery & Replanning)	子任务失败时，是重试、绕行、还是降级？

在实际项目中，planner 不一定是独立模块——它可能嵌入在 Agent 框架的核心循环中（如 LangGraph 的 graph definition、ReAct 的 thought-action-observation 循环），也可能作为一个显式的”规划器 Agent”存在（如 gpt-researcher 的 ChiefEditorAgent / Planner Agent）。

2. 范式对比：Agent 任务规划的主流方案

2.1 范式全景

以下是 Agent 规划领域目前主要的六种范式，按规划抽象层次从低到高排列：



2.2 六种范式的详细对比

(A) ReAct (Reasoning + Acting)

代表实现: LangChain Agent、OpenAI Function Calling、Anthropic Tool Use

ReAct 是当前最广泛使用的 Agent 规划范式。它的核心理念是 **reasoning trace (推理轨迹)** 和 **action (工具调用) 交错进行**：

用户提问 → Thought (分析需求) → Action (调用工具) → Observation (观察结果)
→ Thought (研判结果) → Action (再次调用) → Observation (再次观察)
→ ... → Final Answer

优势: 在线纠错能力强——每步都能根据上一步结果调整；实现简单。 **劣势:** 每一步只看到”上一步的工具结果”，缺乏全局规划视野；长任务容易漂移；工具调用链长度增长时可靠性急剧下降（N步链中每一步的正确率乘积）。

适用场景: 短链工具调用（3-5 步内）、信息检索 + 简单操作。

(B) Plan-and-Execute

代表实现: LangGraph Plan-and-Execute、BabyAGI、AutoGPT

Plan-and-Execute 是”先规划、后执行”的思路：

用户目标 → Planner (生成完整步骤列表/任务列表)
→ Executor (按序执行每步，中间结果累积到 memory)
→ 可选: Replanner (根据执行结果修正后续计划)

优势: 全局规划先行，任务不漂移；天然支持并行子任务。 **劣势:** 初始计划一旦有缺陷，全盘执行方向就会偏；需要额外的 replanning 机制补救；静态计划难以适应动态环境。

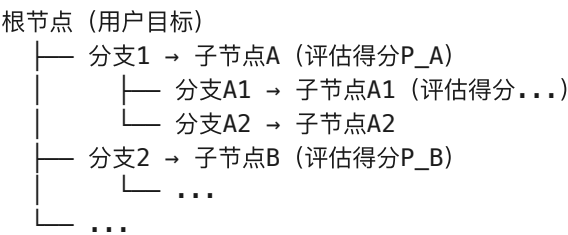
gpt-researcher 的经典模式 (basic/detailed report) 就是 Plan-and-Execute 的变体:
plan_research_outline() 生成子问题列表 → Crawler Agent 并行采集每个子问题的信息 → Publisher 汇总。

适用场景: 目标明确、领域已知的复杂任务（如写调研报告、多步数据处理流水线）。

(C) Tree-of-Thought / Graph-of-Thought (ToT / GoT)

代表实现: 学术原型 (Yao et al. 2023 NeurIPS)、LM 推理优化

ToT 把规划视为 **树搜索问题**:



每个节点代表一个中间思考状态，LLM 同时生成多个候选分支，然后用 BFS/DFS 或 beam search 选择最佳路径。GoT 更进一步，把树泛化为图，允许节点间聚合与交叉引用。

gpt-researcher 的 Deep Research 模式本质上是 ToT 的简化版: 用广度优先的树状搜索探索子主题树，每个子主题是一个树节点，可配置 depth（深度）和 breadth（广度）控制探索范围，branches 间并行处理并共享上下文。

优势: 多路径探索，不容易陷入局部最优；天然适合需要对比分析的场景。 **劣势:** Token 成本与分支数成正比（广度 x 深度 = $O(b^d)$ ）；评估机制依赖 LLM 自身判断，可能不可靠。

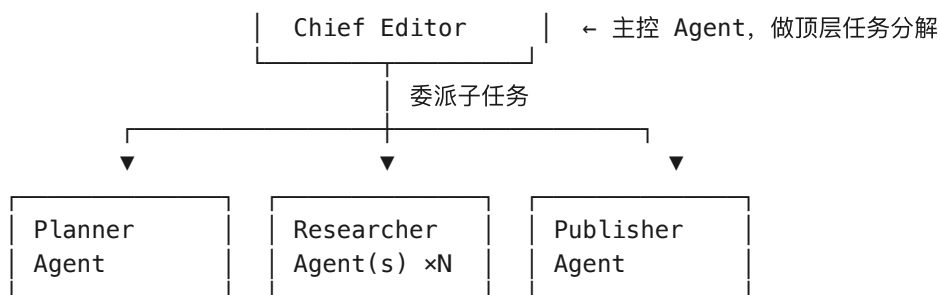
适用场景: 需要多角度分析、有多个竞争性方案的决策型任务。

(D) Multi-Agent 协作规划

代表实现: AutoGen (Group Chat)、CrewAI (Role-Based)、LangGraph (Multi-Agent Graph)、gpt-researcher multi_agents 模式

Multi-Agent 规划把”一个大脑”拆成”一群专家”，每个 Agent 负责自己的子任务：





gpt-researcher 的 multi_agents 模式: ChiefEditorAgent 做顶层调度, 创建 Planner Agent (生成研究大纲 + 子问题) → 多个 Researcher Agent (并行采集信息) → Reviewer Agent (审核) → Reviser Agent (修改) → Publisher Agent (汇总成报告), 形成完整的研究流水线。

优势: 分工明确, 每个 Agent 专注一个子任务, prompt 可以高度特化; 天然适合需要不同专业知识的复杂任务。 **劣势:** Agent 间通信开销大; 编排复杂度高; 调试困难 (哪个 Agent 出问题难定位)。

适用场景: 需要多专业领域知识协作的复杂任务 (深度研究报告、代码审查 + 修复、多轮谈判)。

(E) 代码/DSL 驱动的规划

代表实现: LangGraph (用 Python StateGraph 定义执行图)、DSPy (编程式优化提示链)

Code-Driven Planning 把规划逻辑从 LLM 的”思考”搬到代码层:

```

# LangGraph 风格: 显式定义状态机
class ResearchState(TypedDict):
    query: str
    subtopics: List[str]
    research_data: List[dict]
    report: str

workflow = StateGraph(ResearchState)
workflow.add_node("plan", plan_node)
workflow.add_node("research", research_node)
workflow.add_node("write", write_node)
workflow.add_edge("plan", "research")
workflow.add_edge("research", "write")
workflow.add_conditional_edges("research", decide_next, {
    "continue": "research",
    "write": "write"
})

```

gpt-researcher 的 LangGraph 模式使用这种方式。

优势: 确定性、可调试、可测试；执行路径可预测。 **劣势:** 灵活性受限——不易处理 LLM 层面才能做出的语义级决策；需要开发者预设所有可能路径。

(F) 端到端隐式规划 (E2E Implicit Planning)

代表实现: OpenAI Deep Research Agent、Claude Code Agent (内部循环)

这类 Agent 不对外暴露规划步骤——用户只看到输入和输出。规划过程隐藏在 LLM 自身的内部思考链 (Chain-of-Thought) 中：

用户输入 → [LLM 内部思考 + 工具调用循环, 黑盒] → 最终输出

这类系统通常使用**专有微调模型 + 强化学习 (RL)** 训练规划能力，是目前商业产品的主流路线。

优势: 用户无感知复杂度；可能比手工设计的规划更优 (经过 RL 优化)。 **劣势:** 黑盒、不可解释、不可定制；依赖专有模型，开源生态难以复现。

2.3 范式选择指南

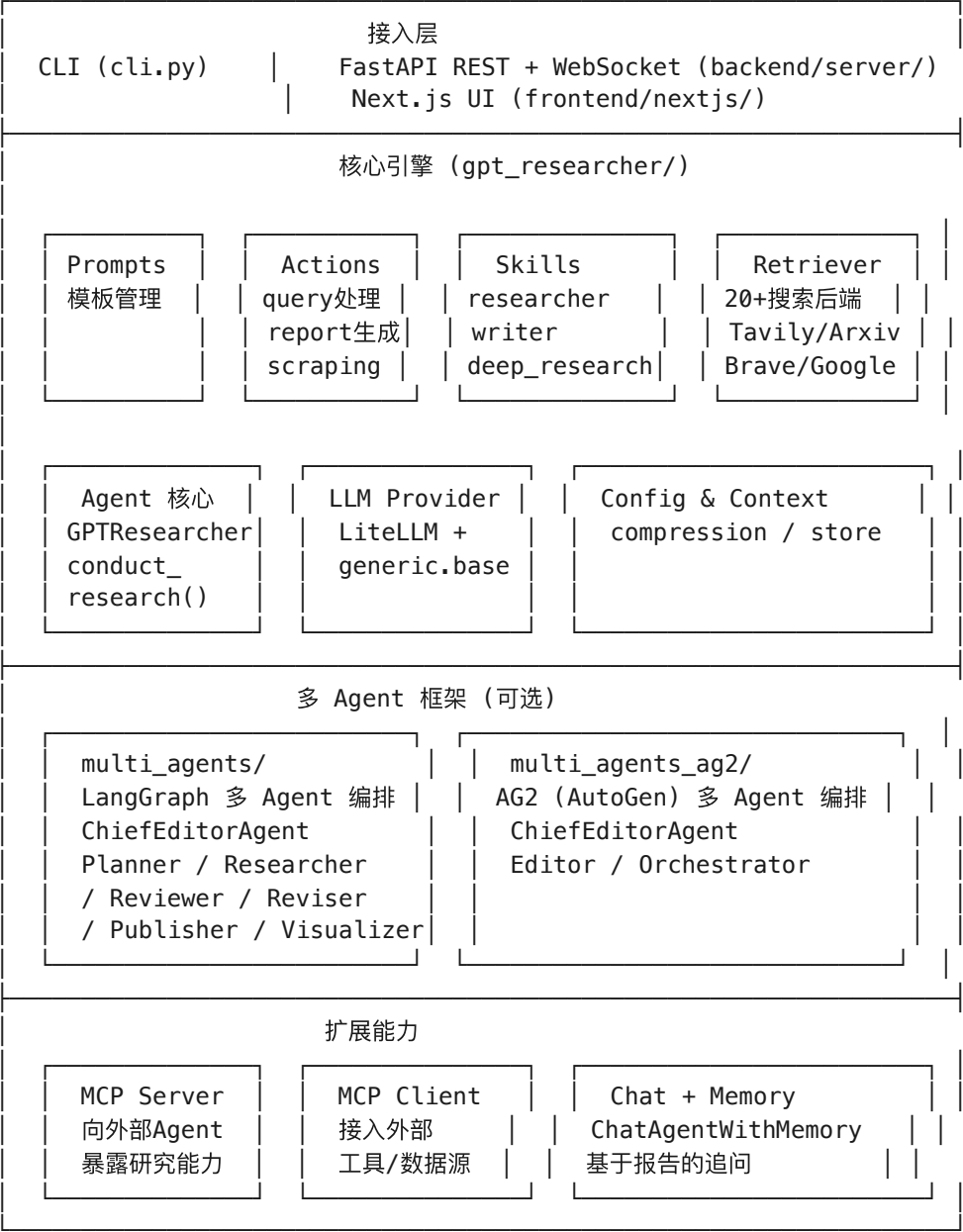
场景特征	推荐范式	原因
目标明确，信息采集类任务	Plan-and-Execute	静态计划即可，不需要动态调整
多工具短链调用	ReAct	开销小，在线纠错
需要多角度分析/对比	Tree-of-Thought	同时探索多路径
需要多专业领域协作	Multi-Agent	角色分工提升质量
需要严格可复现	Code/DSL-Driven	确定性执行
开放式探索	E2E Implicit	灵活度最高但不可控

gpt-researcher 的设计亮点在于**同时提供上述范式的多种变体** (详见第 3 节)，用户可根据研究深度和成本预算选择。

3. gpt-researcher 深度分析

3.1 架构总览

gpt-researcher 是一个**单体仓库但多模式**的自主研究框架，核心架构分为三层：



代码规模: 212 个 Python 文件 + 70 个 TypeScript 文件，核心引擎 `gpt_researcher/` 模块 101 个成员紧密耦合 (cluster cohesion 0.58)，入口点 21 个。

3.2 三种研究模式对比

`gpt-researcher` 提供了三种粒度的研究模式，对应不同的规划策略：

维度	Basic Report	Detailed Report	Deep Research
文件	<code>basic_report.py</code>	<code>detailed_report.py</code>	<code>deep_research/main.py</code>

维度	Basic Report	Detailed Report	Deep Research
规划方式	LLM 一次生成子查询列表	LLM 一次生成子主题大纲	树状递归搜索
并行度	子查询间并行爬取	子主题间并行爬取	树的每层 branches 间并行
搜索深度	单层	单层（可迭代）	可配置 depth × breadth
耗时	~30s	~1-2min	~5min（默认 depth=2, breadth=4）
成本	~\$0.01	~\$0.05-0.10	~\$0.40（o3-mini）
适用场景	快速事实查询	结构化主题报告	学术级深度调研
引用质量	基础	较全	最完整

3.2.1 Basic Report 模式

Basic 是 gpt-researcher 最初的核心模式，采用经典的 **Plan-and-Execute** 范式：

- 1. plan_research_outline(query)
 - LLM 生成一组子研究问题（research questions）
 - 例如 query="电动车电池技术趋势" → ["固态电池最新进展", "锂离子电池成本下降曲线", "钠离子电池产业化现状"]
- 2. 每个 Crawler Agent 独立爬取
 - 对每个子问题：search → scrape N URLs → summarize
 - 所有子问题并行执行
- 3. generate_report() 汇总
 - LLM 将所有子问题的摘要聚合为完整报告

关键代码入口：gpt_researcher/agent.py 的 conduct_research() → get_subtopics() → construct_subtopics()；gpt_researcher/actions/query_processing.py 的 plan_research_outline()。

3.2.2 Detailed Report 模式

Detailed 在 Basic 的基础上增加了**迭代深度**：

- 第一轮：生成更详细的研究大纲（outline）
- 第二轮：对大纲的每个 Section 进行子研究
- 对每个 Section 生成子查询 → 并行采集 → 汇总该 Section → 最后整体汇总

这是 **Plan-and-Execute with Iteration**——相当于把 Plan-and-Execute 跑了两轮。

3.2.3 Deep Research 模式

Deep Research 是 gpt-researcher 最具差异化的能力，采用 **Tree-of-Thought** 的**广度优先搜索变体**：

深度研究循环：

```
for depth in 1..max_depth:
    for each subtopic at current depth:
        1. generate_report_for_subtopic() → 生成当前子主题的初步报告
        2. get_subtopics() → LLM 识别出该子主题的更深层子问题
        3. 对下一层子问题进入下一轮循环
    # 同一层的 subtopics 间并行处理，共享上下文
```

最终：将各层报告汇总为完整研究报告

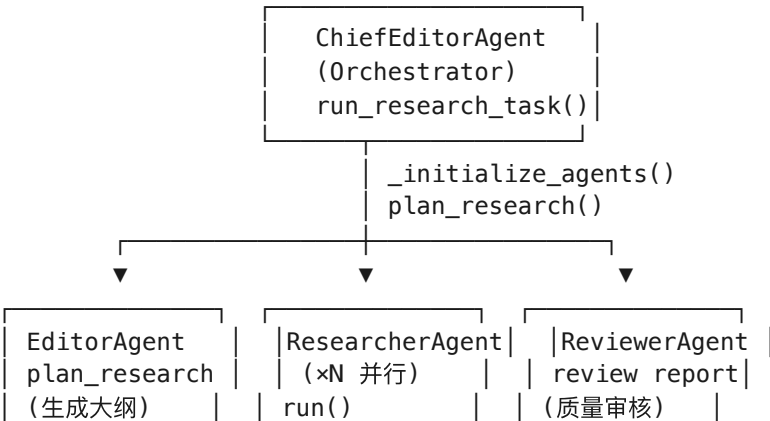
实现位置：gpt_researcher/skills/deep_research.py (ResearchConductor.plan_research() + conduct_research())。

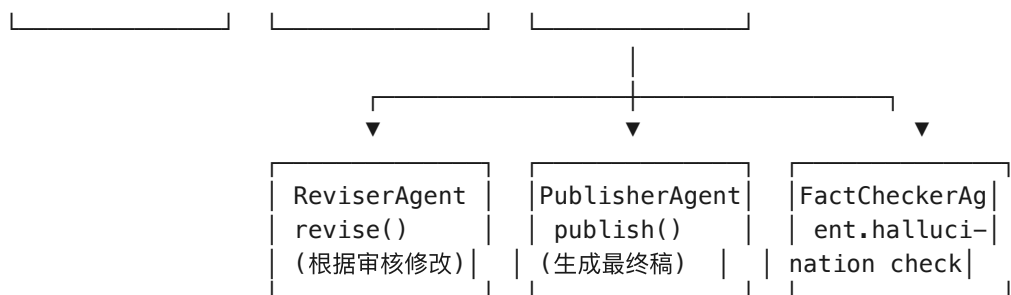
关键参数：- depth：搜索深度（默认 2，即研究主题 → 子主题 → 孙主题） - breadth：每层子问题数（默认 4，即每个主题拆成 4 个子主题） - 总子主题数 $\approx \text{breadth}^{\text{depth}}$ （默认配置下约 $4^2 = 16$ 个子主题）

关于上下文共享（项目开放问题中提到的疑问）：从架构来看，branches 间共享上下文主要通过两个机制：(1) 累积的 research_data 在 conduct_research() 中作为全局状态传递；(2) context/compression.py 中的 ContextCompressor 对历史上下文进行压缩，控制 Token 增长。但的确，如果不加限制地递归，Token 成本会接近 $O(\text{breadth}^{\text{depth}})$ 。

3.3 Multi-Agent 协作模式

当用户选择 Multi-Agent 模式时，gpt-researcher 切换到 LangGraph 或 AG2 编排的**完整研究流水线**：





Agent 角色分工 (multi_agents/agents/ 目录, 共 9 个 Agent 类):

Agent	职责	对应 Planner 子问题
ChiefEditorAgent / Orchestrator	顶层任务调度、初始化 Agent 集群	任务分解
EditorAgent	plan_research() 生成研究大纲和子问题	任务分解
ResearcherAgent	并行搜索、爬取、摘要 (×N)	工具选择与调度
ReviewerAgent	审核研究报告质量、指出不足	错误恢复
ReviserAgent	根据审核意见修改报告	错误恢复/重规划
PublisherAgent	汇总研究结果, 生成最终结构化报告	上下文管理
FactCheckerAgent	事实核查 (hallucination detection)	上下文管理
HumanAgent	人类介入审阅 (可选)	-
VisualizerAgent	生成可视化图表 (可选)	-

两套编排框架: - multi_agents/ — 基于 LangGraph 图编排, 支持 LangSmith 可观测性追踪 - multi_agents_ag2/ — 基于 AG2 (AutoGen) 群聊编排, 代码更精简 (仅 Editor + Orchestrator 两个 Agent 类)

3.4 核心规划循环详解

无论是哪种模式, gpt-researcher 的核心规划逻辑都在 GPTResearcher.conduct_research() 中:

```
# gpt_researcher/agent.py (伪代码)
class GPTResearcher:
    def conduct_research(self):
        # 1. 生成研究计划 (子问题列表)
        subtopics = self.get_subtopics() # → construct_subtopics()

        # 2. 并行执行每个子任务
        for subtopic in subtopics:
            # 2a. 搜索
            search_results = self.retriever.search(subtopic.query)
            # 2b. 爬取
            scraped_data = [scraper.scrape(url) for url in search_results]
            # 2c. 摘要
            summary = llm.summarize(scraped_data)
            # 2d. 累积到 research_data
            self.research_data.append(summary)

        # 3. 生成报告
        report = self.write_report()
        return report
```

实际代码中，`get_subtopics()` 会调用 `construct_subtopics()` (`gpt_researcher/utils/llm.py`)，后者使用 LLM 根据用户查询和上下文生成子主题 JSON（包含子问题标题和任务描述）。`Subtopics` 是一个 Pydantic 模型 (`gpt_researcher/utils/validators.py`)，确保生成格式有效。

3.5 代码架构洞察

从 codebase-memory 分析的 2811 个节点和 7487 条边中，有关 planner 的核心发现：

关键模块边界 (调用频次排名)：1. `agents` → `scraper` (31 次调用)：Crawler Agent 大量依赖网页抓取 2. `retrievers` → `scraper` (25 次调用)：检索模块也需要刮取结果页 3. `report_type` → `agent` (15 次调用)：不同报告类型 (`basic/detailed/deep`) 都调用核心 Agent 4. `agents` → `actions` (12 次调用)：Multi-Agent 模式调用共享的 `actions` 模块

架构分层：- **entry layer**: `agents/`，`report_type/`，`skills/` — 高层业务逻辑，只向外调用 - **core layer**: `agent.py`，`actions/`，`prompts/`，`scraper/`，`server/` — 高扇入，被多处依赖 - **internal layer**: `retrievers/`，`cli/`，`evals/` — 低扇入，相对独立

核心 Agent (`agent.py`) 的扇入 = 23，是项目中被调用最多的模块——印证了它作为规划中枢的角色。

4. Planner 与 Sandbox / Memory / Gateway 的交叉

Planner 不是孤岛。它的规划质量和安全性与 Runtime 的其他子系统紧密耦合：

4.1 Planner – Sandbox 交叉

核心问题: Planner 决定”调用什么工具”，但”工具在哪里执行”是 Sandbox 的决定。

交叉点	问题	gpt-researcher 现状	前沿方向
工具执行安全	Planner 调用的工具可能访问敏感资源（文件系统、网络）	无沙箱隔离，scraper 直接在进程内执行网络请求	OpenShell 提供凭据隔离 + 多后端沙箱驱动，Agent 看不到 API Key
资源配额	Planner 的并行子任务会消耗计算/网络资源	通过 asyncio 并行，无显式配额控制	Agent 沙箱项目（agent-sandbox, CubeSandbox）提供资源隔离
GPU 访问	需要推理的 planner 子任务能否用 GPU？	无 GPU 调度	OpenShell 的 VFIO GPU 直通机制

关键洞察: 当前 gpt-researcher 的工具执行（网页搜索、爬取、LLM 调用）都在同一进程中运行，**没有沙箱隔离**。如果 planner 决策需要执行”危险的”工具（如代码执行、数据库写入），就需要 sandbox 提供安全边界。OpenShell 的架构是这一方向的代表——它将沙箱执行、凭据注入网关和 GPU 直通组合成一个统一的 Runtime。

4.2 Planner – Memory 交叉

核心问题: Planner 做决策需要”记住”哪些上下文？每次规划是否需要从零开始？

交叉点	问题	gpt-researcher 现状	前沿方向
跨步状态传递	多步规划中，上一步的结果如何传给下一步？	research_data 列表累积中间结果	Mem0 的三层记忆分类（用户/会话/Agent）+ 自动记忆提取
长期记忆	用户上次研究的上下文能否被下次规划复用？	后端 SQLAlchemy 持久化研究任务 + 基于报告的追问	Mem0 的 mem0.add() / mem0.search()

交叉点	问题	gpt-researcher 现状	前沿方向
			API，跨会话记忆自动提取
上下文压缩	多步规划的累积上下文导致 Token 爆炸	context/compression.py 的 ContextCompressor	Letta 的 Stateful Agent 记忆管理
记忆冲突	新旧信息冲突时 planner 如何选择？	无显式冲突处理	Mem0 的记忆更新与合并策略

关键洞察：gpt-researcher 的记忆管理还处于‘会话级’——backend/memory/ 目录只有 research.py 和 draft.py 做基础的 CRUD 持久化，gpt_researcher/memory/embeddings.py 做向量存储。缺乏 Mem0 那种自动从对话中提取结构化记忆、跨会话复用、关系图建模的能力。这意味着如果用户连续做三次相关主题的研究，planner **不会自动复用前两次的研究发现**——除非用户显式上传之前的报告作为文档输入。

4.3 Planner – Gateway 交叉

核心问题：当 planner 拆分出多个子任务需要并行执行时，这些子任务的调用如何路由和负载均衡？

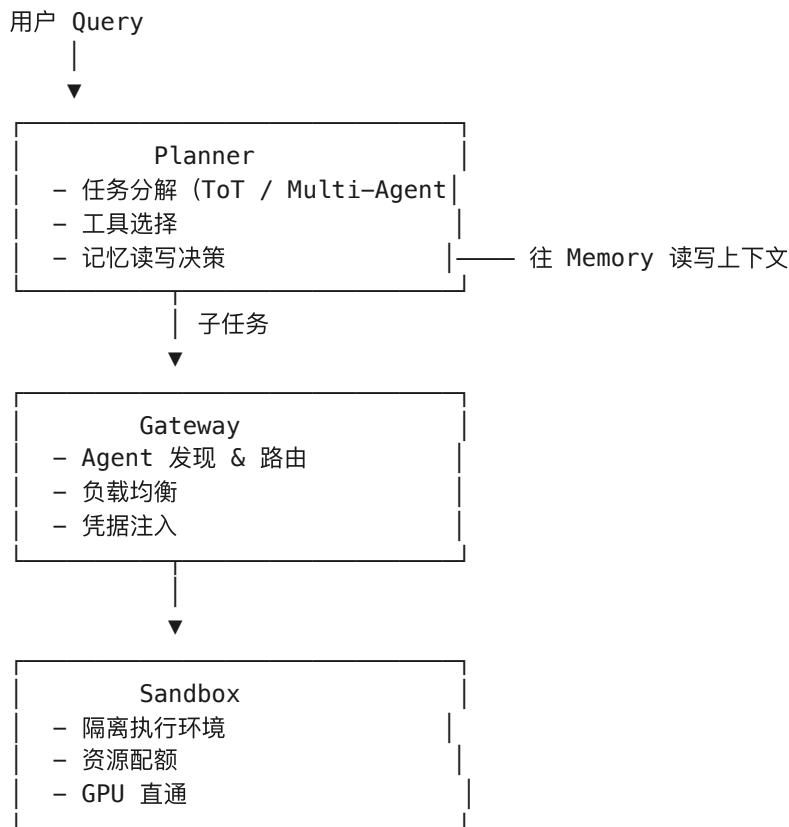
交叉点	问题	gpt-researcher 现状	前沿方向
Agent 发现	Planner 如何知道有哪些”下级 Agent”可用？	硬编码在代码中 (ChiefEditorAgent._initialize_agents())	AgentGateway 的 A2A 协议 Agent Card 注册发现
子任务路由	Planner 的子任务如何转发到对应执行者？	同一进程内直接调用	AgentGateway 的 Agent-to-Agent 路由和负载均衡
认证与权限	子 Agent 调用外部 API 时的凭据管理？	通过环境变量配置 API Key（不安全）	OpenShell 的凭据代理（SIGv4 签名 / token grant）
多协议	Planner 能否调度非 LangGraph/AG2 框架的 Agent？	仅支持 LangGraph 或 AG2 二选一	AgentGateway 的 HTTP/gRPC/A2A 多协议统一接入

关键洞察：gpt-researcher 目前的 Agent 编排是**进程内紧耦合**——ChiefEditorAgent 通过 _initialize_agents() 直接创建本进程内的 Agent 对象。这与 AgentGateway 倡导的”松耦合

Agent Mesh” 形成对比。在 AgentGateway 的架构中，每个 Agent 是独立部署的服务，planner 通过网关发现和路由。松耦合的优势是可扩展性和异构 Agent 支持，代价是额外的网络开销。

4.4 跨子系统协同的理想图景

综合以上分析，一个理想的 Agent Runtime Planner 需要：



在这个图景中，planner 是决策层（“做什么”），gateway 是路由层（“交给谁做”），sandbox 是执行层（“在哪里安全地做”），memory 是持久层（“做过什么”）。gpt-researcher 的当前实现将这三层都压缩在单进程内，但它的三阶段流水线和多 Agent 编排模式已经体现了这种分层的雏形。

5. 开放问题与前沿方向

5.1 gpt-researcher 特定的开放问题

1. **Token 成本线性增长**：Deep Research 的 tree search 中，branches 间共享上下文的具体实现机制是什么？是否存在更好的压缩/选择性共享策略（如只传递高层发现而非全量上下

文) ?

2. **LangGraph vs AG2 兼容性差异**: 两套多 Agent 框架间切换时, Agent 行为是否一致? 团队若后续引入多 Agent 编排, 应优先评估哪个框架? (建议: LangGraph 更成熟、生态更广; AG2 代码更精简但功能更少)
3. **MCP 集成的深度**: gpt-researcher 的 MCP 支持还不够成熟 (MCP Server/Client 代码在 `gpt_researcher/mcp/` 目录, 共 5 个文件), 能否在 Agent Gateway 架构中作为可插拔的研究服务模块复用?

5.2 领域级开放问题

4. **Plan Validation (规划验证)**: 当前所有范式 (ReAct / Plan-and-Execute / ToT) 都没有对”规划本身是否正确”进行独立验证。LLM 生成的计划可能包含逻辑错误、遗漏关键步骤或不切实际的假设。是否需要独立的 Plan Validator?
5. **Dynamic Replanning (动态重规划)**: Plan-and-Execute 的静态计划容易在执行中途变得不适用。当前的补救方案 (Replanner Agent) 仍然依赖 LLM 的判断。是否有更结构化的重规划策略 (如基于执行轨迹的自动化方案修改)?
6. **Cross-Agent Memory (跨 Agent 共享记忆)**: Multi-Agent 模式中, 不同 Agent 的记忆如何共享? 谁负责记忆的读写权限? Mem0 的方案是”所有 Agent 共享一个记忆层”, 但不同 Agent 可能有不同的记忆访问需求。
7. **Planning Cost Optimization (规划成本优化)**: ToT 和 Multi-Agent 范式的 Token 成本远高于 ReAct。能否用”小模型规划 + 大模型执行”降低成本? 或者缓存常见规划模板?
8. **Human-in-the-Loop Planning (人机协同规划)**: gpt-researcher 的 HumanAgent 提供了一个简单的人类介入点, 但人机协同规划的粒度——是”审核整份研究计划”, 还是”逐个子问题确认”——的最佳平衡点尚未明确。

6. 参考资料

- [gpt-researcher GitHub](#) — 28k+ Stars, 自主 AI 研究 Agent
- [gpt-researcher summary](#) — KG 中的项目摘要
- [ReAct: Synergizing Reasoning and Acting in Language Models](#) — Yao et al., 2022. ReAct 范式原始论文
- [Tree of Thoughts: Deliberate Problem Solving with Large Language Models](#) — Yao et al., 2023 NeurIPS. ToT 范式原始论文

- [Graph of Thoughts: Solving Elaborate Problems with Large Language Models](#) — Besta et al., 2023. GoT 范式原始论文
- [STORM: Assisting in Writing Wikipedia-like Articles From Scratch](#) — Shao et al., 2024. Stanford STORM, gpt-researcher Deep Research 模式的重要灵感来源
- [LangGraph Documentation](#) — gpt-researcher 的多 Agent 编排运行时
- [AG2 \(AutoGen\)](#) — gpt-researcher 的第二多 Agent 框架
- [AgentGateway](#) — Agent 间通信网关, 与 planner 的路由决策相关
- [Mem0](#) — Agent 记忆层, 与 planner 的上下文管理相关
- [OpenShell](#) — Agent 沙箱运行时, 与 planner 的工具执行安全相关